Week 4: Analytical Inverse Kinematics

EE0849 Introduction to Robotics - Detailed Notes

Basri Erdogan

Table of contents

1 Introduction to Inverse Kinematics

1.1 The Inverse Kinematics Problem

Inverse kinematics (IK) is the problem of finding the joint variables that achieve a desired end-effector pose. This is the inverse of forward kinematics.

Problem Statement:

- **Given:** Desired end-effector pose $T_{desired}$ (position and orientation)
- **Find:** Joint variables $q = [\theta_1, \theta_2, ..., \theta_n]^T$

$$q = f^{-1}(T_{desired})$$

1.2 Forward vs Inverse Kinematics

Aspect	Forward Kinematics	Inverse Kinematics
Input	Joint angles	End-effector pose
Output	End-effector pose	Joint angles
Solution	Always unique	May have 0, 1, or many
Equations	Linear (matrix)	Nonlinear (trig)
Difficulty	Straightforward	Challenging

1.3 Why is IK Difficult?

1. **Non-uniqueness:** Multiple joint configurations can produce the same end-effector pose
2. **Non-existence:** Some poses may be outside the robot's workspace
3. **Singularities:** Certain configurations cause loss of degrees of freedom
4. **Nonlinear equations:** Transcendental equations without general closed-form solutions

1.4 Solution Approaches

Approach	Description	Pros	Cons
Geometric	Use trigonometry and geometry	Intuitive, fast	Limited to simple robots
Algebraic	Solve polynomial equations	General	Complex derivation
Numerical	Iterative optimization	Works for any robot	May not converge, slow

This week focuses on **geometric (analytical)** solutions.

2 Workspace Analysis

2.1 Definitions

Reachable Workspace: The set of all points in space that the end-effector can reach with at least one orientation.

Dexterous Workspace: The set of all points that can be reached with any arbitrary orientation.

$$W_{dexterous} \subseteq W_{reachable}$$

2.2 2-Link Planar Arm Workspace

For a 2-link planar arm with lengths L_1 and L_2:

- **Maximum reach:** $r_{max} = L_1 + L_2$ (fully extended)
- **Minimum reach:** $r_{min} = |L_1 - L_2|$ (fully folded)

The workspace is an **annulus** (ring) between these two radii.

2.3 Reachability Condition

A point (x, y) is reachable if and only if:

$$|L_1 - L_2| \leq \sqrt{x^2 + y^2} \leq L_1 + L_2$$

Always check reachability before attempting IK!

```
def is_reachable(x, y, L1, L2):
    r = np.sqrt(x**2 + y**2)
    return abs(L1 - L2) <= r <= L1 + L2
```

3 Geometric IK for 2-Link Planar Arm

3.1 Problem Setup

Given: - Link lengths: L_1, L_2 - Target position: (x, y)

Find: - Joint angles: θ_1, θ_2

3.2 Step 1: Solve for θ_2

From forward kinematics:

$$x = L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2)$$
$$y = L_1 \sin\theta_1 + L_2 \sin(\theta_1 + \theta_2)$$

Squaring and adding:

$$x^2 + y^2 = L_1^2 + L_2^2 + 2L_1 L_2 \cos\theta_2$$

Solving for $\cos\theta_2$:

$$\cos\theta_2 = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1 L_2}$$

3.3 Step 2: Two Solutions for θ_2

Since $\sin^2\theta_2 + \cos^2\theta_2 = 1$:

$$\sin\theta_2 = \pm\sqrt{1 - \cos^2\theta_2}$$

This gives two solutions:

- **Elbow Up:** $\sin\theta_2 > 0 \rightarrow \theta_2 > 0$
- **Elbow Down:** $\sin\theta_2 < 0 \rightarrow \theta_2 < 0$

```
# Elbow up
s2_up = np.sqrt(1 - c2**2)
theta2_up = np.arctan2(s2_up, c2)

# Elbow down
s2_down = -np.sqrt(1 - c2**2)
theta2_down = np.arctan2(s2_down, c2)
```

3.4 Step 3: Solve for θ_1

Using the geometry:

$$\theta_1 = \text{atan2}(y, x) - \text{atan2}(L_2\sin\theta_2, L_1 + L_2\cos\theta_2)$$

Or more compactly:

$$\theta_1 = \text{atan2}(y, x) - \alpha$$

where $\alpha = \text{atan2}(L_2 s_2, L_1 + L_2 c_2)$

3.5 The atan2 Function

The `atan2(y, x)` function returns the angle whose tangent is y/x, correctly handling all four quadrants:

- Returns values in $(-\pi, \pi]$
- Handles $x = 0$ cases
- **Always use `atan2` instead of `atan` in robotics!**

```
# Wrong - only works in quadrant I
theta = np.arctan(y / x)  # Fails when x = 0

# Correct - works everywhere
theta = np.arctan2(y, x)
```

3.6 Complete Algorithm

```
import numpy as np

def ik_2link(x, y, L1, L2, elbow='up'):
    """
    Inverse kinematics for 2-link planar arm.

    Parameters:
        x, y: Target position
        L1, L2: Link lengths
        elbow: 'up' or 'down'
```

```
    Returns:
        theta1, theta2: Joint angles (radians)
    """
    # Step 1: Check reachability
    c2 = (x**2 + y**2 - L1**2 - L2**2) / (2*L1*L2)
    if abs(c2) > 1:
        raise ValueError("Point unreachable")

    # Step 2: Solve for theta2
    if elbow == 'up':
        s2 = np.sqrt(1 - c2**2)
    else:
        s2 = -np.sqrt(1 - c2**2)
    theta2 = np.arctan2(s2, c2)

    # Step 3: Solve for theta1
    alpha = np.arctan2(L2*s2, L1 + L2*c2)
    theta1 = np.arctan2(y, x) - alpha

    return theta1, theta2
```

3.7 Verification

Always verify IK solutions with forward kinematics:

```
def fk_2link(theta1, theta2, L1, L2):
    x = L1*np.cos(theta1) + L2*np.cos(theta1 + theta2)
    y = L1*np.sin(theta1) + L2*np.sin(theta1 + theta2)
    return x, y

# Verify: FK(IK(target)) should equal target
x_check, y_check = fk_2link(theta1, theta2, L1, L2)
assert np.allclose([x_check, y_check], [x, y])
```

4 Solution Selection

4.1 Multiple Solutions

For a 2-link arm, most reachable points have **two** IK solutions: - Elbow up - Elbow down

How do we choose?

4.2 Selection Criteria

1. **Minimize motion:** Choose solution closest to current configuration
2. **Joint limits:** Reject solutions that violate physical limits
3. **Obstacle avoidance:** Choose collision-free configuration
4. **Singularity avoidance:** Stay away from singular poses
5. **Task requirements:** Some tasks prefer specific configurations

4.3 Joint Limits

```python
def ik_with_limits(x, y, L1, L2, q_limits):
    """Return all valid IK solutions within joint limits."""
    solutions = []

    for elbow in ['up', 'down']:
        try:
            theta1, theta2 = ik_2link(x, y, L1, L2, elbow)

            # Check limits
            if (q_limits[0][0] <= theta1 <= q_limits[0][1] and
                q_limits[1][0] <= theta2 <= q_limits[1][1]):
                solutions.append((theta1, theta2, elbow))
        except ValueError:
            pass  # Unreachable

    return solutions
```

4.4 Singularities

Singular configurations occur when the robot loses a degree of freedom:

For 2-link arm: - $\theta_2 = 0$: Fully extended (can't move radially) - $\theta_2 = \pm\pi$: Fully folded (can't move radially)

At singularities: - Jacobian becomes singular - Small Cartesian motions require infinite joint velocities

5 6-DOF Robots with Spherical Wrist

5.1 Kinematic Decoupling

For robots with a **spherical wrist** (where axes 4, 5, 6 intersect at a point):

1. **Position problem:** Find $\theta_1, \theta_2, \theta_3$ to place the wrist center
2. **Orientation problem:** Find $\theta_4, \theta_5, \theta_6$ for desired orientation

This **decouples** the 6-DOF problem into two 3-DOF problems!

5.2 Wrist Center Calculation

The wrist center is offset from the end-effector:

$$\mathbf{p}_{wc} = \mathbf{p}_{ee} - d_6 \cdot \mathbf{z}_6$$

Where: - $\mathbf{p}_{ee}$: End-effector position (from desired pose) - d_6: Wrist offset (from DH parameters) - $\mathbf{z}_6$: Approach direction (third column of rotation matrix)

5.3 Position IK (Joints 1-3)

With the wrist center (x_{wc}, y_{wc}, z_{wc}) known:

Joint 1:

$$\theta_1 = \text{atan2}(y_{wc}, x_{wc})$$

Joints 2-3: Use 2-link IK in the vertical plane containing the wrist center.

5.4 Orientation IK (Joints 4-6)

Once $\theta_1, \theta_2, \theta_3$ are known:

1. Compute R_3^0 using forward kinematics
2. Extract R_{ee}^0 from the desired pose
3. Calculate $R_6^3 = (R_3^0)^T R_{ee}^0$
4. Decompose R_6^3 into ZYZ Euler angles $\rightarrow \theta_4, \theta_5, \theta_6$

5.5 Multiple Solutions

A 6-DOF robot with spherical wrist can have up to **8 IK solutions**:

- 2 for shoulder (left/right or front/back)
- 2 for elbow (up/down)
- 2 for wrist (flip/no-flip)

$$2 \times 2 \times 2 = 8$$

6 Industrial Robot Configurations

6.1 FANUC Configuration Strings

FANUC robots use three-character strings to specify configurations:

Position	Character	Meaning
1	N/F	**N**o-flip / **F**lip (wrist)
2	U/D	Elbow **U**p / **D**own
3	T/B	**T**op (front) / **B**ack (arm)

Example: `NUT` = No-flip, Up, Top

6.2 Configuration Selection in Practice

When programming industrial robots:

1. **Specify configuration** for each waypoint
2. **Maintain configuration** during continuous motion when possible
3. **Plan configuration changes** carefully (may require large motions)

6.3 Python Example

```python
def parse_fanuc_config(config_str):
    """Parse FANUC NUT/FDB configuration string."""
    return {
        'flip': config_str[0] == 'F',
        'elbow_up': config_str[1] == 'U',
        'front': config_str[2] == 'T'
    }

# Example
config = parse_fanuc_config('NUT')
print(config)
# {'flip': False, 'elbow_up': True, 'front': True}
```

7 Python Implementation

7.1 Using roboticstoolbox

```python
from roboticstoolbox import DHRobot, RevoluteDH
from spatialmath import SE3
import numpy as np

# Create 2-link robot
robot = DHRobot([
    RevoluteDH(d=0, a=1.0, alpha=0),
    RevoluteDH(d=0, a=0.5, alpha=0)
], name='2-Link')

# Define target pose
T_target = SE3.Trans(0.8, 0.6, 0)

# Solve IK (numerical)
sol = robot.ikine_LM(T_target)
print(f"Solution: {np.degrees(sol.q)}")
print(f"Success: {sol.success}")
```

7.2 IK Methods in roboticstoolbox

Method	Function	Notes
Levenberg-Marquardt	`ikine_LM()`	Robust, general
Gauss-Newton	`ikine_GN()`	Faster
Newton-Raphson	`ikine_NR()`	Classic
Analytical	`ikine_a()`	If available

7.3 Initial Guess Matters

Numerical IK methods are sensitive to the initial guess:

```python
# Different initial guesses may yield different solutions
sol1 = robot.ikine_LM(T_target, q0=[0, 0])      # May find elbow up
sol2 = robot.ikine_LM(T_target, q0=[0, -1])    # May find elbow down
```

8 Summary

8.1 Key Formulas

2-Link IK:

$$\cos\theta_2 = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}$$

$$\theta_1 = \text{atan2}(y, x) - \text{atan2}(L_2 s_2, L_1 + L_2 c_2)$$

Reachability:

$$|L_1 - L_2| \leq r \leq L_1 + L_2$$

8.2 Key Concepts

1. IK finds joint angles for desired end-effector pose
2. Multiple solutions typically exist
3. Always check reachability first
4. Verify solutions with FK
5. Spherical wrist enables kinematic decoupling
6. Configuration strings specify which solution to use

8.3 Common Mistakes

1. Using `atan` instead of `atan2`
2. Forgetting to check reachability
3. Not verifying with FK
4. Ignoring joint limits
5. Assuming unique solution exists

9 Practice Problems

9.1 Problem 1

For a 2-link arm with $L_1 = 1.2$ m, $L_2 = 0.8$ m: a) What is the reachable workspace? b) Find both IK solutions for point $(1.5, 0.5)$ c) Verify using FK

9.2 Problem 2

A point is at distance $r = L_1 + L_2$ from the origin. How many IK solutions exist? Why?

9.3 Problem 3

Implement a function that returns the IK solution closest to the current configuration:

```
def ik_nearest(x, y, L1, L2, q_current):
    """Return IK solution closest to current configuration."""
    # Your implementation here
    pass
```